과제 #6

M1522.006700 확장형 고성능 컴퓨팅 (001)

2021-15531 컴퓨터공학부 김인호
November 26, 2025

1 Matrix Multiplication Challenge

본 과제에서는 단일 노드 내에 장착된 여러 개의 GPU를 활용하여 행렬곱 연산 $C = A \times B$의 성능을 극대화하는 코드를 구현하였다. MPI를 이용한 다중 노드 통신 오버헤드를 배제하고, 단일 노드 내 4개의 GPU 자원을 효율적으로 스케줄링하는 호스트 코드 최적화와 연산 효율을 극대화한 커널 최적화에 집중하였다.

- **병렬화 방식 (Multi-GPU Strategy)**
  이 과제에서는 4대의 계산 노드를 사용하는 것이 허용되었지만, 실제로 4대의 계산 노드를 모두 활용하가 위해 MPI를 사용하여 테스트해본 결과 오히여 1대의 계산 노드만 사용했을 때보다 성능이 낮아지는 결과를 확인할 수 있었다. 이는 A와 B 행렬이라는 거대한 데이터를 MPI의 API를 활용해서 노드끼리 전송하는 시간이, 각 노드가 행렬곱을 계산하는 시간보다 훨씬 커서 여러 노드를 사용하는 효과가 사라지기 때문으로 추측된다. MPI를 사용할 때 최대한 오버헤드를 숨길 수 있는 구조를 고안할 수도 있지만, 이번 과제에서는 1개의 노드만 사용하되, 저번보다 커널를 더욱 최적화하는 방식을 사용하였다.

  따라서 이번 과제에서 사용한 병렬화 방식은 저번 과제와 크게 달라지지 않았다. 간략하게 정리하면 아래와 같다.

  - GPU 4대에 A 행렬을 행 단위로 균등하게 나누어 전달하여, 각각이 계산해야 할 C 행렬을 행 단위로 분배하였다. 4대의 GPU는 동시에 행렬곱 연산을 병렬적으로 수행한다.
  - 특히 호스트(CPU)에서, CUDA의 stream과 비동기 API를 활용하여 각 GPU에 명령을 내리도록 하였다. 호스트가 GPU의 응답을 기다리지 않고 빠르게 모든 GPU에 명령을 내림으로써, 각 GPU는 행렬 데이터의 복사와 커널 실행 등의 명령을 병렬적으로 수행한다.
  - 각각의 GPU는 계산해야 할 C 행렬을 청크(`CHUNK`) 단위로 나누어 각 스레드에게 분배하였다. 각 스레드는 자신이 맡은 C 행렬의 부분에 대한 행렬곱 연산을 병렬적으로 수행한다.

- **성능 최적화를 위해 적용한 방법 및 고려 사항**
  위에서 설명한 병렬화 이외에도 아래의 방법들을 활용하여 성능 최적화를 진행하였다.

  - **Shared Memory Tiling**: 전역 메모리 접근을 줄이기 위해 행렬 A와 B를 `TILE` 크기의 블록 단위로 공유 메모리에 적재하여 재사용한다. 공유 메모리는 전역 메모리보다 접근이 빠르므로 메모리 접근 속도에서 이익을 본다.
  - **Register Tiling**: 공유 메모리보다더 더 빠른 레지스터를 활용하기 위해, 스레드 당 연산량을 늘려 레지스터 단에서 데이터를 재사용한다. 기존에는 각 스레드가 C 행렬의 한 원소를 계산하였지만, 이번 코드에서는 각 스레드가 `CHUNK` $\times$ `CHUNK` 크기의 C 행렬 서브 타일을 계산하도록 하여, 한 번 로드한 A, B 데이터를 레지스터 내에서 최대한 재활용하고 연산의 밀도를 높였다.
  - **Double Buffering**: 공유 메모리 타일을 A, B에 대해 각각 2개씩 할당하여 사용한다. 현재 타일을 사용하여 연산하는 동시에 다음 타일을 전역 메모리에서 로드하여 연산 시간과 메모리 접근 시간 겹치게 하여 시간을 단축시킨다.

- **Loop Unrolling**: C 행렬의 결괏값을 계산할 때, CUDA에서 지원하는 `#pragma unroll` 지시어를 사용하여 반복문의 제어 오버헤드를 줄이고 명령어 파이프라인 효율을 높였다.

위와 같은 병렬화와 최적회 방법을 적용하고 $\texttt{TILE} = 16, \texttt{CHUNK} = 6$으로 둔 뒤 $M = 65535, N = K = 4096$에서 성능을 측정한 결과, 22285.521376 GFLOPS의 결과를 얻었다.

- **`matmul.cu`의 각 부분 설명**
  `matmul_initialize` 함수는 CUDA 디바이스를 개수를 세고 각 GPU의 이름을 확인한 뒤, 각 GPU에 할당할 행렬 A, C의 행 구간을 설정하고 행렬 데이터를 저장할 공간을 각 GPU의 전역 메모리에 할당한다. 아래의 CUDA API가 쓰인다.

  - `cudaGetDeviceCount`: 사용할 수 있는 CUDA 디바이스의 개수를 알려준다.
  - `cudaGetDeviceProperties`: 디바이스의 정보들(이름, 사양 등)을 가져온다.
  - `cudaSetDevice`: 호스트 스레드에서 명령에 사용할 디바이스를 설정한다.
  - `cudaMalloc`: 디바이스(GPU)에 지정한 크기만큼의 메모리를 할당한다.
  - `cudaStreamCreate`: 비동기 처리를 위한 스트림을 만든다.

  `matmul` 함수는 각각의 GPU에서 계산할 A 행렬의 일부분과 B 행렬 전체를 각 GPU로 복사하고, 그리드와 블록의 차원을 설정하여 `matmul_kernel`을 실행시킨 뒤, 해당 GPU가 담당한 C 행렬의 일부분에 결과를 담는다. 여기에 쓰이는 API 중 앞에서 설명하지 않은 것들을 아래에서 설명한다.

  - `cudaMemcpyAsync`: 호스트와 디바이스 간에 (인자를 통해 방향 설정) 데이터를 복사하는 명령을 스트림에 넣고 비동기적으로 실행시킨다.
  - `cudaDeviceSynchronize`: 디바이스가 실행을 완료할 때까지 기다린다.

  `matmul_kernel` 함수는 각 GPU가 담당한 작업을 스레드별로 수행하는 부분이다. 위에서 설명한 타일링, 더블 버퍼링 등을 활용하여 행렬곱 연산을 수행한다.

  `matmul_finalize` 함수는 각각의 GPU에 대해 행렬 데이터를 담기 위해 할당했던 모든 메모리와 스트림 자원을 해제한다.

  - `cudaFree`: 디바이스에 할당된 메모리를 해제(free)한다.
  - `cudaStreamDestroy`: 생성했던 비동기 스트림을 제거한다.

- **성능 실험 및 결과 분석** 본 실험에서는 $M = 65535, N = K = 4096$ 크기의 행렬을 사용하여 성능을 측정하였다. 성능 지표는 GFLOPS로 표기하였다.

  Multi-GPU 활용
  단일 노드 내에서 사용하는 GPU의 개수를 늘려가며 성능 향상을 측정하였다. 데이터가 각 GPU에 분할되어 병렬로 처리되므로 GPU 수에 비례한 성능 향상을 기대할 수 있다.

GPU 개수	1	2	3	4
성능(GFLOPS)	5632.689463	11252.972591	16851.505279	22312.684567

  실험 결과, GPU 개수가 늘어남에 따라 성능이 거의 비례해서 증가하는 모습을 보였다.

  Double Buffering 활용
  메모리 접근 시간을 숨기기 위한 Double Buffering의 적용 유무에 따른 성능 차이를 비교하였다. 코드에서 다음 타일을 로드하는 부분과 행렬곱 연산을 수행하는 부분 사이에 `__syncthreads()`를 넣으면 더블 버퍼링을 하지 않는 효과를 줄 수 있다. 이렇게 했을 때 성능은 20882.224914 GFLOPS가 나왔다.

  따라서 Double Buffering을 적용했을 때, 연산과 메모리 로드가 중첩되어 성능이 약 6.7% 향상됨을 확인할 수 있었다.

  레지스터 및 공유 메모리 타일링
  레지스터와 공유 메모리에 대한 타일링에 관여하는 `TILE`, `CHUNK` 변수를 조절하며 성능을 비교하였다.

CHUNK \ TILE	8	16
4	14533.870364	18072.888638
5	13525.450745	17904.504368
6	20633.712413	22312.684567
7	18912.103083	21279.028980
8	20361.262141	21911.622178

(성능은 GFLOPS 단위로 표기)

청크와 타일 크기가 커질수록 전반적으로 성능이 상승하지만, 타일 크기가 16일 때 청크 크기가 6보다 커지면 공유 메모리나 레지스터 자원의 포화로 인해 성능이 소폭 하락한다고 해석할 수 있다.